



Especificação Funcional

Especificação Funcional (EF)

[DEM-999666] EST-DEM-999666-AGV - CUSTOMER PERCEPTION

Informações do documento

Cliente	CPFL
Projeto	CUSTOMER PERCEPTION
Demanda / DM	DEM-999666
Sistema / módulo	AGV
Torre / Diretoria	CPFL AMS
Empresas / unidades impactadas:	CPFL
Versão	1.0
Data da criação	12/07/2026
Elaborado por	NTT DATA gustavo.berbert@gmail.com
Status	Aprovado

Classificação: Uso interno / Confidencial

Controle do documento

Versão	Data	Autor	Descrição da alteração	Aprovador
1.0	12/07/2026	gustavo.berbert@gmail.com	Criação inicial do documento	-

1. Objetivo

Esta especificação técnica detalha o desenho de arquitetura de software para a implementação do painel **Conta Fácil 2.0** e do motor de mensageria associado. O objetivo primordial deste projeto é disponibilizar aos clientes da CPFL um dashboard interativo de detalhamento de faturas ("Conta Fácil Didática") de forma resiliente, escalável e segura, além de prover um canal de comunicação direta via notificações (SMS, WhatsApp, Push e E-mail).

A solução arquitetural é baseada no ecossistema **Acquia Cloud Platform** hospedando o CMS **Drupal 9/10**, atuando estritamente como um **Frontend Leve / Backend For Frontend (BFF)**. Todas as integrações com os sistemas legados e repositórios de dados corporativos da CPFL serão mediadas exclusivamente pelo Barramento de Integração Corporativo (**Oracle Integration Cloud - OIC e/ou SAP Process Orchestration - SAP PO**), eliminando conexões ponto a ponto (P2P) entre a camada web e os sistemas transacionais/analíticos da companhia.

O escopo técnico deste documento abrange:

- Desenho detalhado de componentes e fluxos de dados ponta a ponta.
- Modelagem física de banco de dados (tabelas de fallback, auditoria e entidades do Drupal).
- Contratos OpenAPI estruturados para todas as APIs internas e externas.
- Mecanismos de resiliência e concorrência baseados na Drupal Queue API com tratamento estrito de timeout.
- Estratégia de testes, requisitos de segurança, matriz de riscos e planos de mitigação.

2. Visão Técnica da Solução

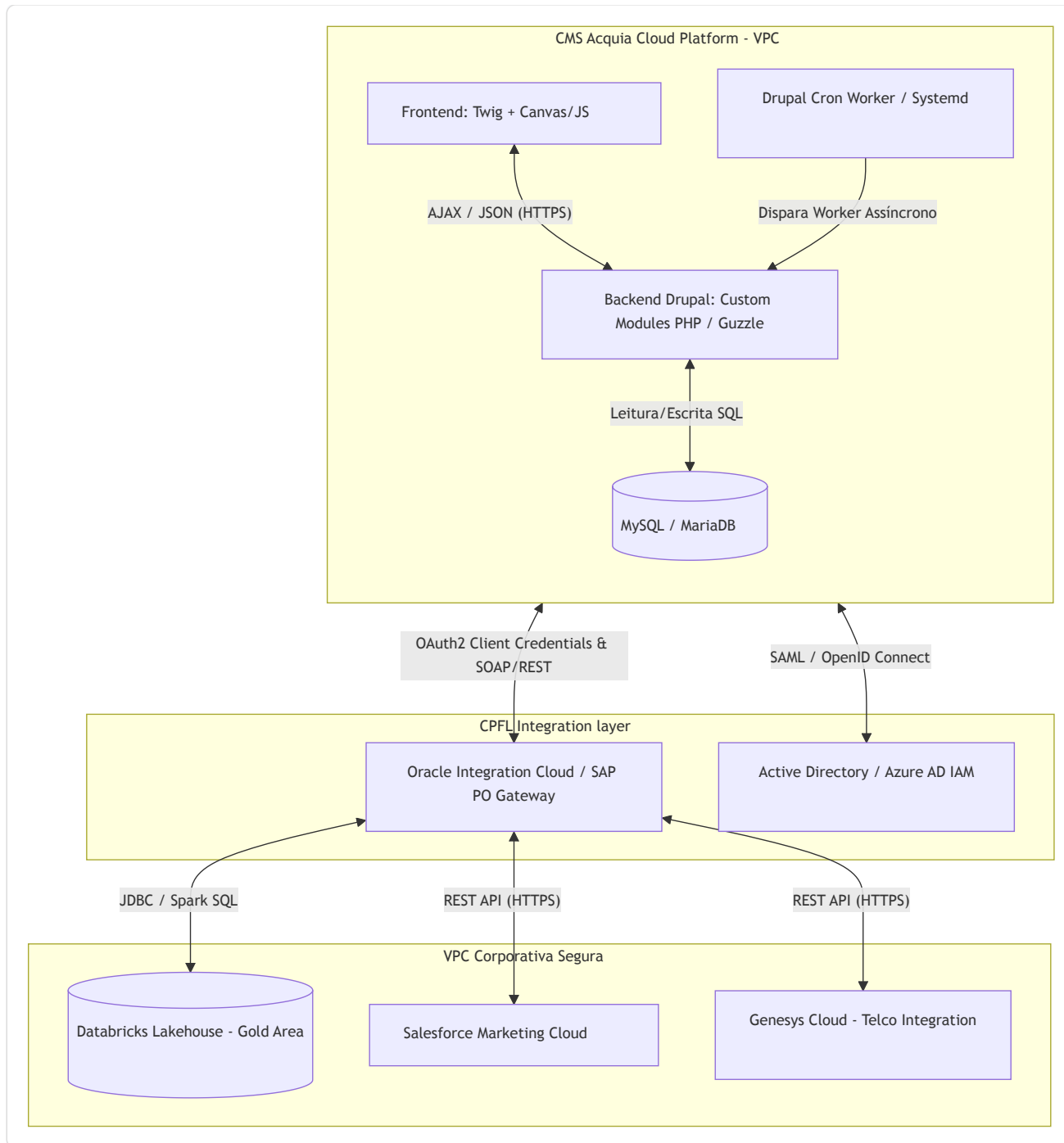
A solução afasta-se de arquiteturas monolíticas tradicionais de CMS ao adotar o padrão arquitetural **BFF (Backend for Frontend)** desacoplado dentro do próprio Drupal. Toda a lógica de computação analítica, processamento tarifário e consolidação tributária é delegada ao **Databricks (Lakehouse)** e exposta de forma simplificada por meio do OIC. O CMS Acquia é responsável unicamente pelas seguintes responsabilidades técnicas:

1. **Apresentação e Roteamento:** Renderização estática com hidratação dinâmica de dados via chamadas AJAX assíncronas do lado do cliente para rotas internas do Drupal PHP.
 2. **Orquestração e Consumo de APIs:** Mediação de chamadas RESTful seguras com autenticação baseada em tokens OAuth2 gerenciados de forma persistente.
 3. **Gestão de Filas de Contingência (Fallback):** Uso de armazenamento relacional local temporário para reenvio e escalonamento de mensagens de notificação quando os tempos de resposta dos canais finais (Salesforce Marketing Cloud e Genesys Cloud) excederem o SLA crítico estabelecido de **2000 milissegundos**.
 4. **Governança e Auditoria Local:** Registro não mutável em banco relacional local de cada acesso a dados sensíveis de faturas para fins de compliance com a LGPD (Lei Geral de Proteção de Dados).
-

3. Arquitetura

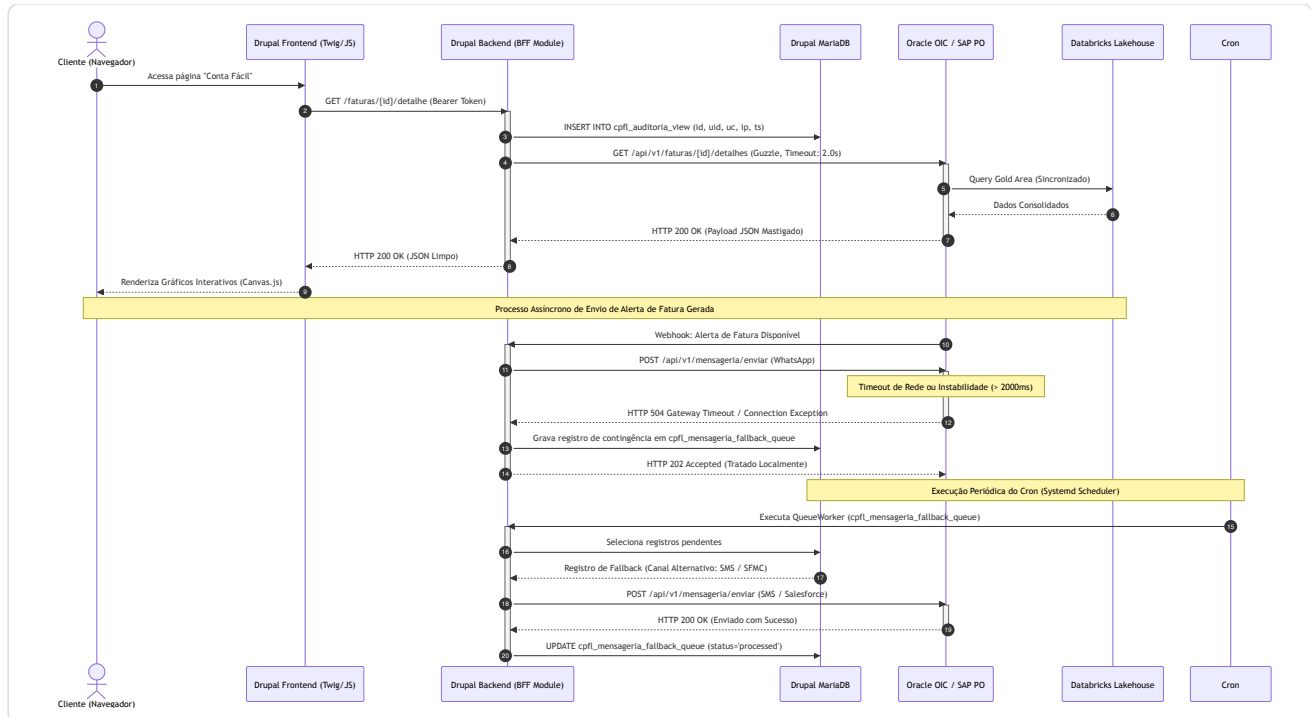
3.1. Topologia de Componentes e Integração

O diagrama abaixo detalha a topologia de implantação física e lógica da solução no ecossistema híbrido da CPFL:



3.2. Fluxo de Sequência de Execução (Visualização de Fatura e Fallback)

O fluxo abaixo apresenta a jornada do usuário ao acessar o detalhamento da fatura, explicitando o tratamento de erro e o enfileiramento de fallback assíncrono:



3.3. Pipelines de Build e Deploy (CI/CD)

O ciclo de vida do código-fonte segue fluxos de entrega contínua rigorosamente separados com base na governança de infraestrutura da CPFL:

Esteira de Deploy Acquia (Drupal):

O repositório git é hospedado na infraestrutura do **Acquia Code Studio**.

Static Application Security Testing (SAST): PHP_CodeSniffer com padrões Drupal e validações de segurança do PhpStan Security Rules.

O build é empacotado compilando os assets do frontend (Node.js/SASS/Webpack) e instalando as dependências de backend via Composer (`composer install --no-dev`).

O deploy do artefato finalizado é orquestrado de forma automatizada via **Acquia Cloud Pipelines** percorrendo as ramificações de ambiente: `dev` (desenvolvimento), `stage` (homologação) e `prod` (produção).

Esteira de Integrações e Componentes Externos (.NET / OIC):

Toda alteração em proxies de barramento OIC ou pipelines de carga Databricks segue as esteiras preexistentes da CPFL baseadas em **Azure DevOps**, sem intersecção de deploy com a camada de apresentação Acquia.

4. Detalhes Técnicos e Alterações

4.1. Camada de Apresentação (Frontend)

Template de Apresentação Principal: Criação do arquivo `block--conta-facil-didatico.html.twig` dentro do diretório do tema customizado corporativo (`web/themes/custom/cpfl_theme/templates/block/`). Este template define a estrutura semântica HTML5 para exibir os blocos didáticos (Custo de Energia, Distribuição, Encargos Setoriais, Tributos).

Biblioteca Gráfica Visual: Inclusão e carregamento de biblioteca de desenho via JavaScript (exclusivamente declarada no arquivo `cpfl_theme.libraries.yml`). Utilização de **Chart.js v4.x** via arquivo JS customizado `js/conta-facil-charts.js` para renderizar o gráfico histórico de evolução de consumo (últimos 12 meses).

Gestão Dinâmica de Preferências (Opt-ins): Sobrescrita do formulário padrão de perfil do usuário (`user_profile_form`) usando a hook do Drupal `hook_form_ALTER()` . Os campos booleanos de consentimento de notificação serão renderizados como chaves dinâmicas (HTML5 switches estilizados via CSS).

4.2. Camada de Backend e Serviços (Módulos PHP Customizados)

4.2.1. Módulo `cpfl_conta_facil`

Expõe o endpoint REST interno que o frontend consome assincronamente via JavaScript.

Rota Interna: `GET /faturas/{id}/detalhe`

Classe Controller: `\Drupal\cpfl_conta_facil\Controller\ContaFacilController`

Exemplo de Payload de Resposta (JSON esperado pelo frontend):

```
{
  "id_fatura": "100988776611",
  "codigo_uc": "998877",
  "mes_referencia": "2023-11",
  "valor_total": 352.40,
  "data_vencimento": "2023-12-10",
  "grupo_faturamento": "B1",
  "composicao_valores": {
    "custo_energia": {
      "valor": 120.50,
      "percentual": 34.2
    },
    "distribuicao_transporte": {
      "valor": 95.80,
      "percentual": 27.2
    },
    "encargos_setoriais": {
      "valor": 45.10,
      "percentual": 12.8
    },
    "tributos": {
      "icms": 68.40,
      "pis_cofins": 22.60,
      "total": 91.00,
      "percentual": 25.8
    }
  },
  "historico_consumo": [
    {"mes": "2023-10", "valor_kwh": 310, "valor_reais": 340.20},
    {"mes": "2023-09", "valor_kwh": 290, "valor_reais": 315.00},
    {"mes": "2023-08", "valor_kwh": 320, "valor_reais": 348.50}
  ]
}
```

4.2.2. Módulo cpfl_mensageria

Serviço responsável pela integração com o OIC para envio de notificações e ativação automática do sistema de contingência.

Classe Service: \Drupal\cpfl_mensageria\Service\MensageriaClient

Assinatura do Método Principal:

```
public function enviarMensagem(string $uc, string $canal, array $dadosMensagem): bool;
```

Contrato da API Externa (OIC Gateway) consumido pelo Drupal:

Endpoint: POST https://oic-gateway.cpfl.com.br/api/v1/mensageria/enviar

Headers:

Content-Type: application/json

Authorization: Bearer <JWT_OAUTH2_TOKEN>

Payload de Envio (Request JSON):

```
{
  "origem": "ACQUIA_CMS",
  "codigo_uc": "998877",
  "canal_comunicacao": "WHATSAPP",
  "template_id": "tpl_fatura_disponivel_v2",
  "destinatario": "+5519999999999",
  "parametros_dinamicos": [
    {
      "chave": "nome_cliente",
      "valor": "João da Silva"
    },
    {
      "chave": "mes_referencia",
      "valor": "Novembro/2023"
    },
    {
      "chave": "valor_fatura",
      "valor": "352,40"
    }
  ]
}
```

```
"chave": "codigo_barras",  
"valor": "836200000035 524001380006 310102023118"  
}  
}  
}
```

Códigos de Retorno Esperados:

- 200 OK ou 202 Accepted : Sucesso de processamento.
- 400 Bad Request : Payload inconsistente com os esquemas validados.
- 401 Unauthorized / 403 Forbidden : Token expirado ou sem escopo adequado.
- 500 / 503 / 504 : Falha do provedor ou timeout de rede (Gera Fallback).

4.3. Processamento Assíncrono (Jobs & Queues)

Definição de Queue Worker: Implementação de um plugin do Drupal anotado com `@QueueWorker` que herda de `QueueWorkerBase`.

ID do Plugin: `cpfl_mensageria_fallback_processor`

Frequência de Execução: Disparado nativamente no ciclo do Cron do sistema operacional (Systemd Daemon chamando o comando `drush cron` a cada 5 minutos).

Fluxo Técnico do Worker: O worker processará registros da tabela `cpfl_mensageria_fallback_queue` cujo status seja igual a `pending`.

- Lê a mensagem e tenta re-envio mudando o canal originalmente configurado para o canal alternativo definido nos metadados do usuário (ex: se o canal WhatsApp retornou erro persistente de timeout, o worker força o roteamento da mensagem via SMS tradicional através do Salesforce Cloud).
- Registra o incremento de tentativas (`tentativas++`).
- Em caso de sucesso, marca o status como `processed`.
- Se atingir o limite estrito de **3 tentativas** sem resposta do gateway, muda o status para `failed` e gera um log crítico no subsistema de logs do Drupal.

4.4. Logs e Tratamento de Erro

Padrão de Logs: Uso estrito da classe `\Drupal\Core\Logger\LoggerChannelFactoryInterface` injetada via injeção de dependência nos serviços customizados.

Comportamento diante de Exceções de Rede (Guzzle): Todos os blocos de chamada externa serão envolvidos em escopos `try/catch` para capturar exceções especializadas do Guzzle:

```
try {  
    $response = $this->httpClient->post($url, $options);  
} catch (\GuzzleHttp\Exception\ConnectException $e) {  
    // Tratamento estrito para timeout de conexão (< 2000ms)  
    $this->logger->error('Timeout de conexão com o barramento OIC no envio da UC {uc}. Causa: {error}', [  
        'uc' => $uc,  
        'error' => $e->getMessage(),  
    ]);  
    $this->enqueueFallback($uc, $payload, 'CONNECTION_TIMEOUT');  
} catch (\GuzzleHttp\Exception\RequestException $e) {  
    // Tratamento para HTTP status codes 4xx/5xx de erro  
    $this->logger->critical('Falha de protocolo na requisição da UC {uc}. Status Code: {status}. Causa: {error}', [  
        'uc' => $uc,  
        'status' => $e->getResponse() ? $e->getResponse()->getStatusCode() : 'Sem Resposta',  
        'error' => $e->getMessage(),  
    ]);  
    $this->enqueueFallback($uc, $payload, 'PROTOCOL_ERROR');  
}
```

5. Dados e Banco de Dados

O banco de dados relacional (MySQL/MariaDB) hospedado no ecossistema de infraestrutura PaaS da Acquia atuará estritamente como base de suporte estrutural, auditoria transacional de segurança e controle de fila assíncrona.

5.1. Novas Estruturas Criadas via API de Schema do Drupal (`hook_schema`)

Estas tabelas serão criadas programaticamente no momento de ativação dos respectivos módulos PHP customizados (`cpfl_conta_facil.install` e `cpfl_mensageria.install`).

Tabela: `cpfl_mensageria_fallback_queue`

Gerencia mensagens retidas na camada local de persistência devido a indisponibilidades do Barramento OIC.

Campo	Tipo de Dado	Restrição	Índices / Chaves	Descrição
id	INT UNSIGNED	AUTO_INCREMENT	PRIMARY KEY	Identificador único transacional do fallback.
uc_fatura	VARCHAR(20)	NOT NULL	-	Código da Unidade Consumidora da fatura.
payload_json	LONGTEXT	NOT NULL	-	Payload JSON completo pronto para ser submetido ao OIC.
canal_tentado	VARCHAR(15)	NOT NULL	-	Nome do canal que sofreu timeout/erro (ex: WHATSAPP).
status	VARCHAR(15)	NOT NULL	INDEX (idx_status_tentativa)	Status atual: pending , processed , failed .
tentativas	TINYINT	DEFAULT 0	INDEX (idx_status_tentativa)	Contador de tentativas de reprocessamento (Max: 3).
timestamp_criacao	INT UNSIGNED	NOT NULL	-	Época UNIX da criação do registro de contingência.
timestamp_atualizacao	INT UNSIGNED	NOT NULL	-	Época UNIX da última alteração de estado física.

-- Representação em sintaxe SQL padrão para referência técnica:

```
CREATE TABLE cpfl_mensageria_fallback_queue (  
  id INT UNSIGNED AUTO_INCREMENT NOT NULL,  
  uc_fatura VARCHAR(20) NOT NULL,  
  payload_json LONGTEXT NOT NULL,  
  canal_tentado VARCHAR(15) NOT NULL,  
  status VARCHAR(15) NOT NULL,  
  tentativas TINYINT DEFAULT 0 NOT NULL,  
  timestamp_criacao INT UNSIGNED NOT NULL,  
  timestamp_atualizacao INT UNSIGNED NOT NULL,  
  PRIMARY KEY (id),  
  INDEX idx_status_tentativa (status, tentativas)  
) ENGINE=InnoDB DEFAULT CHARSET=utf8mb4;
```

Tabela: cpfl_auditoria_view

Garante a rastreabilidade rígida exigida por LGPD em relação à exibição dos dados financeiros de fatura de clientes finais.

Campo	Tipo de Dado	Restrição	Índices / Chaves	Descrição
id	INT UNSIGNED	AUTO_INCREMENT	PRIMARY KEY	Chave primária de auditoria.
uid	INT UNSIGNED	NOT NULL	INDEX (idx_uid_uc)	Chave estrangeira ligando ao identificador do usuário (users.uid).
uc_acessada	VARCHAR(20)	NOT NULL	INDEX (idx_uid_uc)	Unidade consumidora consultada no evento de visualização.
ip_address	VARCHAR(45)	NOT NULL	-	IP que originou a requisição HTTP (IPv4 ou IPv6).
timestamp	INT UNSIGNED	NOT NULL	-	Época UNIX do exato momento da requisição física.

-- Representação em sintaxe SQL padrão para referência técnica:

```
CREATE TABLE cpfl_auditoria_view (  
  id INT UNSIGNED AUTO_INCREMENT NOT NULL,  
  uid INT UNSIGNED NOT NULL,  
  uc_acessada VARCHAR(20) NOT NULL,  
  ip_address VARCHAR(45) NOT NULL,  
  timestamp INT UNSIGNED NOT NULL,  
  PRIMARY KEY (id),
```

```
INDEX idx_uid_uc (uid, uc_acessada)
) ENGINE=InnoDB DEFAULT CHARSET=utf8mb4;
```

5.2. Alteração em Entidades Nativas do Drupal

Para acomodar as preferências de comunicação de notificações de maneira robusta, serão adicionados de forma programática campos no escopo da entidade de usuário nativa (`User`) do Drupal Core via scripts de configuração de esquema:

1. `field_optin_whatsapp` (Tipo: `Boolean` | Valor Padrão: `0`)
2. `field_optin_sms` (Tipo: `Boolean` | Valor Padrão: `0`)
3. `field_optin_push` (Tipo: `Boolean` | Valor Padrão: `0`)
4. `field_optin_email` (Tipo: `Boolean` | Valor Padrão: `1`)

5.3. Procedures Relacionais

Nota Técnica: Não aplicável. Toda manipulação, queries relacionais e controle transacional no banco de dados seguirão estritamente o padrão corporativo de boas práticas para Drupal API (`\Drupal::database()`), utilizando abstrações orientadas a objetos seguras contra injeção de SQL.

6. Estratégia de Testes Técnicos

Esta seção estabelece o plano rigoroso de validação de qualidade, garantindo que as implementações respeitem o desempenho, resiliência e integridade estipulados.

Tipo de Teste	Escopo da Validação Técnica	Ferramental Sugerido	Critério de Aceitação / SLA
Unitário	Validação das lógicas de estruturação interna de dados da classe controller e do cálculo dos dados do gráfico histórico de faturas. Cobertura do parsing de contratos JSON.	PHPUnit (Nativo do Drupal Core testing).	Mínimo de 85% de cobertura de código (linhas de código útil de lógica de negócio).
Integração	Simulação ponta a ponta do envio de mensagens ativando a rotina de Queue API no banco MariaDB local, simulando sucesso de gravação e mudança de status.	PHPUnit Kernel Test Suites.	Sucesso absoluto na leitura e escrita local de filas em banco sem concorrência de deadlocks.
API	Testes de contratos, chamadas externas reais e de simulações com mockups em relação ao barramento OIC de desenvolvimento.	Postman / Newman CLI.	Validação exata da estrutura do schema OpenAPI dos payloads JSON retornados com HTTP 200.
Performance	Injeção massiva de acessos simultâneos concorrentes na rota interna do Drupal para simulação de pico de cargas.	k6 / JMeter.	Tempo de resposta interna do BFF de menos de 300ms (excluindo tempo do OIC). Tempo total integrado dentro do SLA global de 2000ms sob carga contínua de 500 requisições simultâneas por segundo.
Segurança	Análise estática de código-fonte (SAST) para identificar vulnerabilidades estruturais no PHP (OWASP Top 10, SQL Injection, XSS) e controle de autenticação em APIs.	Acquia Code Studio / SonarQube.	Zero vulnerabilidades de severidade Alta/Crítica apontadas no relatório de estática de código.
Regressão	Verificação de integridade no processamento padrão de rotinas de login unificado (SAML) de usuários e painéis preexistentes.	Selenium / Cypress.	Desempenho idêntico ao histórico de produção anterior para as funcionalidades intocadas da agência virtual.

7. Impactos, Riscos e Dependências Técnicas

7.1. Matriz de Riscos de Engenharia (RT-0xx)

Código	Risco Técnico Mapeado	Impacto	Probabilidade	Mitigação Arquitetural e Plano de Resolução
RT-001	Instabilidade ou indisponibilidade temporária de barramento OIC / SAP PO (Timeout de requisições).	Alta	Média	Implementação do padrão de projeto <i>Graceful Degradation</i> . Se a chamada AJAX do frontend falhar por timeout de 2s, o backend do Drupal captura e envia ao frontend um payload parcial indicando que o detalhamento didático está indisponível, substituindo o bloco de erro por um botão de download direto de PDF alternativo legado.
RT-002	Contenção e travamentos de banco de dados (Deadlock) na tabela <code>cpfl_mensageria_fallback_queue</code> durante execuções do Cron.	Média	Baixa	Utilização estrita de paginação, controle por transação individualizada para cada linha de processamento de fila e definição explícita de <code>LOCK IN SHARE MODE</code> ou bloqueio de cursor no limite de registros processados por lote de loteamento de Cron (<i>Batch Size Limit</i>).
RT-003	Consumo excessivo de memória em rotinas PHP devido a alta volumetria de consumo assíncrono na leitura do banco.	Média	Baixa	Implementação de limpeza explícita de memória volátil do PHP (<code>unset()</code>), liberação manual de conexões com o DB e imposição estrita de diretivas de <code>memory_limit: 512M</code> globais no PHP-FPM gerenciados pela infraestrutura Acquia.

7.2. Dependências Técnicas

Governança de Autenticação corporativa (Azure AD / IAM): A rota de detalhes da fatura requer validação da sessão corporativa e checagem de claims (atributos do usuário) para garantir que a Unidade Consumidora solicitada pertença autenticamente ao usuário conectado. É pré-requisito técnico o token OAuth2 de usuário trafegado pelo cabeçalho HTTP de requisições conter a claim válida e decodificável.

Janelas de Sincronia de Dados (Databricks Gold Layer): O projeto parte do pressuposto absoluto de que as faturas fechadas no sistema SAP comercial já passaram pelo processo ETL de consolidação de cargas e estão devidamente integradas e acessíveis na camada Gold do Databricks com seus valores de composição financeira mastigados no momento exato do acesso do cliente final na agência virtual. Qualquer atraso neste processo ETL de ingestão será invisível ao CMS Acquia e deverá ser mitigado pelas equipes de dados.

7.3. Dependências de Negócio

Sincronia Jurídica de Consentimentos de Notificação (Opt-ins/Opt-outs): Como o Drupal permitirá a alteração desses canais de comunicação, as alterações devem persistir instantaneamente de volta aos ecossistemas corporativos que originam as réguas de relacionamento (Salesforce Cloud Marketing) para evitar o descumprimento legal da vontade expressa do cliente final sob regras rígidas da LGPD.

8. Referências e Anexos Técnicos

Referência de Negócio: Documento de Requisitos de Negócio - Conta Fácil 2.0 (versão de arquitetura funcional e detalhamento de regras fiscais).

Especificações de API e Barramento: Contratos OpenAPI unificados do OIC para faturas didáticas da CPFL.

Políticas Internas da CPFL: Diretiva Corporativa de Segurança da Informação e Privacidade (Manual Técnico de Desenvolvimento de Software Seguro v3.4).

9. Glossário Técnico

Termo / Sigla	Definição e Aplicação Contextualizada no Projeto
Acquia Cloud Platform	Plataforma como Serviço (PaaS) líder em hospedagem cloud para ambientes baseados no CMS Drupal, oferecendo alta performance e alta disponibilidade em nuvem gerenciada.
BFF (Backend for Frontend)	Padrão arquitetural de microserviços/serviços que cria uma camada intermediária de backend especializada em atender estritamente um determinado formato de interface de usuário (frontend).
Databricks Lakehouse	Plataforma de dados unificada que combina o melhor do armazenamento Data Lake com a governança, integridade de transações ACID e organização relacional típica de Data Warehouses.

Termo / Sigla	Definição e Aplicação Contextualizada no Projeto
Guzzle	Biblioteca padrão de requisições HTTP escrita em PHP de nível profissional utilizada nativamente no ecossistema de bibliotecas do núcleo do CMS Drupal.
OIC (Oracle Integration Cloud)	Middleware SaaS corporativo da Oracle atuando no ecossistema CPFL como camada controlada de API Gateway, barramento e orquestração integrada de conectores.
Opt-in / Opt-out	Termo legal de permissão em que um cliente concorda ativamente (Opt-in) ou revoga (Opt-out) a recepção de comunicações e coleta de seus dados de privacidade sob a égide da LGPD.
Queue API (Drupal)	Subsistema de gerenciamento de tarefas assíncronas do Drupal que gerencia armazenamento em tabelas transacionais até que sejam consumidos por rotinas do Cron.
SAST	Static Application Security Testing. Metodologia automatizada de segurança de aplicação usada para analisar o código-fonte antes de sua execução em busca de fraquezas de segurança.
Twig	Motor de renderização de templates robusto, seguro, moderno e extremamente rápido utilizado nativamente na camada visual do Drupal para geração do HTML.
UC (Unidade Consumidora)	Identificador técnico padrão para localizar o ponto exato de conexão de energia de um consumidor na infraestrutura de distribuição da CPFL.

Obrigado